Лабораторные работы по Kubernetes (K3S)

Лабораторная работа №1: Установка и настройка K3S, знакомство с базовыми объектами Kubernetes

Цель работы:

Освоить процесс установки и настройки облегченной версии Kubernetes (K3S), научиться работать с базовыми объектами Kubernetes: Pod, Deployment, Service.

Варианты заданий:

Вариант 1:

1. Установить K3S на локальную машину.
2. Создать простой Pod с контейнером nginx.
3. Создать Deployment с 3 репликами контейнера httpd.
4. Создать Service типа ClusterIP для доступа к Deployment.
5. Проверить работоспособность созданных объектов.

Вариант 2:

1. Установить K3S на локальную машину.
2. Создать Pod с контейнером Python, который выводит текущее время каждые 10 секунд.
3. Создать Deployment с 2 репликами контейнера Redis.
4. Создать Service типа NodePort для доступа к Deployment.
5. Проверить работоспособность созданных объектов.

Вариант 3:

1. Установить K3S на локальную машину.

2. Создать Pod с контейнером PostgreSQL с настроенной переменной окружения для пароля.

3. Создать Deployment с 3 репликами контейнера Nginx с настроенной страницей приветствия.

4. Создать Service типа LoadBalancer для доступа к Deployment.

5. Проверить работоспособность созданных объектов.

Контрольные вопросы:

1. Что такое Kubernetes и какие проблемы он решает?
2. Чем K3S отличается от стандартного Kubernetes?
3. Что такое Pod и почему это базовая единица в Kubernetes?
4. В чем разница между Pod и Deployment?
5. Какие типы Service существуют в Kubernetes и чем они отличаются?
6. Что такое kubectl и какие основные команды вы использовали?
7. Как проверить логи работающего контейнера в Pod?
8. Как получить доступ к приложению, запущенному в кластере Kubernetes?

Лабораторная работа №2: Управление состоянием приложений в Kubernetes

Цель работы:

Освоить методы хранения и управления состоянием приложений в Kubernetes с использованием ConfigMap, Secret, PersistentVolume и PersistentVolumeClaim.

Варианты заданий:

Вариант 1:

1. Создать ConfigMap с конфигурацией для веб-сервера Nginx.
2. Создать Secret с учетными данными для доступа к базе данных.
3. Создать Deployment с контейнером Nginx, который использует созданный ConfigMap для конфигурации.
4. Создать PersistentVolume и PersistentVolumeClaim для хранения данных.
5. Создать Pod с контейнером, который монтирует PersistentVolumeClaim и записывает в него данные.

Вариант 2:

1. Создать ConfigMap с переменными окружения для приложения на Python.
2. Создать Secret с API-ключами для внешнего сервиса.
3. Создать Deployment с контейнером Python, который использует созданные ConfigMap и Secret.
4. Создать StatefulSet с 3 репликами контейнера MongoDB с постоянным хранилищем.
5. Проверить сохранность данных при перезапуске Pod.

Вариант 3:

1. Создать ConfigMap с конфигурационным файлом для MySQL.

2. Создать Secret с паролями для MySQL.

3. Создать Deployment с контейнером WordPress, который использует созданные ConfigMap и Secret.

4. Создать PersistentVolume и PersistentVolumeClaim для хранения данных MySQL.

5. Создать Deployment с контейнером MySQL, который монтирует PersistentVolumeClaim.

Контрольные вопросы

1. Что такое ConfigMap и для чего он используется?
2. Чем Secret отличается от ConfigMap и когда следует использовать Secret?
3. Какие способы монтирования ConfigMap и Secret в контейнер существуют?
4. Что такое PersistentVolume и PersistentVolumeClaim?
5. Какие режимы доступа (Access Modes) существуют для PersistentVolume?
6. Что такое StorageClass и для чего он используется?
7. Чем StatefulSet отличается от Deployment?
8. Как обеспечивается сохранность данных при перезапуске Pod?

Лабораторная работа №3: Сетевое взаимодействие и масштабирование приложений в Kubernetes

Цель работы

Освоить методы организации сетевого взаимодействия между приложениями в Kubernetes, научиться масштабировать приложения и настраивать их отказоустойчивость.

Варианты заданий

Вариант 1:

1. Создать многоуровневое приложение, состоящее из:
   - Frontend (Nginx/React)
   - Backend (Node.js/Express)
   - База данных (MongoDB)
2. Настроить сетевое взаимодействие между компонентами с помощью Service.
3. Настроить Ingress для доступа к frontend извне кластера.
4. Реализовать горизонтальное масштабирование backend с помощью HorizontalPodAutoscaler.
5. Настроить проверки готовности (readiness) и живучести (liveness) для всех компонентов.

Вариант 2:

1. Создать микросервисное приложение, состоящее из:
   - API Gateway (Nginx)
   - Сервис пользователей (Python/Flask)
   - Сервис товаров (Python/Django)
   - База данных (PostgreSQL)
2. Настроить сетевое взаимодействие между сервисами с помощью Service.
3. Настроить Ingress с маршрутизацией запросов к разным сервисам.

4. Реализовать автоматическое масштабирование сервисов с помощью HorizontalPodAutoscaler.

5. Настроить NetworkPolicy для ограничения сетевого взаимодействия между сервисами.

Вариант 3:

1. Создать приложение для обработки сообщений, состоящее из:
   - Frontend (Vue.js)
   - API сервис (Go)
   - Сервис обработки сообщений (Python)
   - Брокер сообщений (RabbitMQ)
2. Настроить сетевое взаимодействие между компонентами с помощью Service.
3. Настроить Ingress с TLS-терминацией для доступа к frontend.
4. Реализовать автоматическое масштабирование сервиса обработки сообщений на основе нагрузки.
5. Настроить ResourceQuota и LimitRange для ограничения ресурсов в namespace.

Контрольные вопросы

1. Что такое Ingress и для чего он используется?
2. Как организовать сетевое взаимодействие между Pod в разных namespace?
3. Что такое HorizontalPodAutoscaler и какие метрики он может использовать для масштабирования?
4. Чем отличаются проверки readiness и liveness? Для чего они нужны?
5. Что такое NetworkPolicy и как с её помощью ограничить сетевой доступ?
6. Как настроить TLS-терминацию в Ingress?
7. Что такое ResourceQuota и LimitRange? Для чего они используются?
8. Какие стратегии обновления (Deployment strategies) существуют в Kubernetes?

9. Как организовать балансировку нагрузки между Pod?

10. Какие инструменты мониторинга можно использовать для отслеживания состояния кластера Kubernetes?